

IA 01 - Git State, History, and Recovery

Lucas Swearingen | CPA 490 - Capstone 1 | Dr. Stiffler

Part 1 — Read Repository State

Claim 1: HEAD identifies the currently checked-out branch in this scenario.

1. Command: `git symbolic-ref --short HEAD` or `git symbolic-ref HEAD`
2. Output: `main` or `refs/heads/main`
3. Explanation: `HEAD` is a symbolic reference that identifies which branch reference is currently checked out rather than pointing directly to a commit object. Executing `git symbolic-ref --short HEAD` inspects this pointer and returns `main`. Furthermore, omitting `--short` demonstrates that `HEAD` currently points to the `refs/heads/main` branch reference. This proves `HEAD` tracks the active branch position in the working tree.

Claim 2: A branch reference identifies a commit.

1. Command: `git rev-parse main` and `git cat-file -t main`
2. Output: `cc72c37b04597b67198ca5c11085492da30f57f7` and `commit`
3. Explanation: A branch is not a directory, but a named reference that stores a 40-character commit object ID. Running `git rev-`

`parse main` resolves the branch reference name `main` directly to its underlying commit hash. This demonstrates that the branch reference acts as a pointer to a specific commit object within the repository graph.

Claim 3: A commit identifies a tree and records its parent relationship.

1. Command: `git cat-file -p main`

2. Output:

```
tree 5288f3716d9fd4a9667c2a1309369c76d3cc2e98
parent bd77aab3cb7a9598dccd6bddfb50fb20e7d52fe7
author Git Lab <git-lab@example.com> 1789396123 -0400
committer Git Lab <git-lab@example.com> 1789396123 -0400
```

Update README

3. Explanation: Inspecting the commit object that `main` points to using `git cat-file -p` shows its internal header fields. The `tree` line identifies a tree object representing the snapshot structure of the directory at that point. The `parent` line records the commit object ID of its predecessor, proving that a commit object explicitly stores pointers backward to its parent ancestry.

Claim 4: The repository contains two branch tips that share earlier history.

1. Command: `git log --graph --oneline --decorate --all`

2. Output:

```
* 0f9a8b4 (feature/validation) Add validation tests
* 04ec968 Add timeout validation
| * cc72c37 (HEAD -> main) Update README
| * bd77aab Add logging
|/
```

```
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

3. Explanation: Executing the `graph log` command displays the graph of all repository references and commits. The output shows two distinct branch references pointing to different tip commit objects, which are `main` and `feature/validation`. Tracing their parent relationships backward shows that both branches diverge from the common ancestor commit, demonstrating that two independent branch tips share earlier history.

Part 2 — Construct a Proposed Commit

Initial states:

1. Command: `git diff`
2. Output:

```
diff --git a/config.properties b/config.properties
index f4c0461..1cd1c5 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
# Core runtime settings
-timeout=30
+timeout=60
  retry.count=3
  retry.delay.ms=250

@@ -10,7 +10,7 @@ connection.keepalive=true
  connection.pool.size=10

# Logging settings
-log.level=INFO
+log.level=DEBUG
  log.console=true
```

```
log.timestamps=true
log.file=app.log
@@ -21,6 +21,6 @@ feature.metrics=false
feature.experimental=false
```

3. Explanation: Executing `git diff` compares the current working tree contents against the index. The output reveals the prepared, unstaged edits in `config.properties`: increasing the timeout, enabling debug logging, and changing the connection limit.

Staging Step:

1. Command: `git add -p config.properties`
2. Output:

```
diff --git a/config.properties b/config.properties
index f4c0461..1cdalc5 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
 # Core runtime settings
-timeout=30
+timeout=60
 retry.count=3
 retry.delay.ms=250

(1/3) Stage this hunk [y,n,q,a,d,j,J,g,/,e,?]? y
@@ -10,7 +10,7 @@ connection.keepalive=true
connection.pool.size=10

 # Logging settings
-log.level=INFO
+log.level=DEBUG
 log.console=true
 log.timestamps=true
 log.file=app.log
(2/3) Stage this hunk [y,n,q,a,d,K,j,J,g,/,e,?]? n
@@ -21,6 +21,6 @@ feature.metrics=false
```

```
feature.experimental=false

# Limits
-max.connections=100
+max.connections=250
max.payload.kb=512
max.queue.depth=25
(3/3) Stage this hunk [y,n,q,a,d,K,g,/,e,]? n
```

3. Explanation: Selectively staged Hunk 1, the timeout change, by typing `y`. Left hunks 2 and 3 unstaged by typing `n`.

Repository Status

1. Command: `git status`
2. Output:

On branch main

Changes to be committed:

```
(use "git restore --staged <file>..." to unstage)
    modified:   config.properties
```

Changes not staged for commit:

```
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    modified:   config.properties
```

3. Explanation: * Executing `git status` summarizes which file paths are different across the working tree, index, and `HEAD`. The exact same file path (`config.properties`) appears simultaneously under both `Changes to be committed` and `Changes not staged for commit` because Git tracks blobs and hunks rather than just a list of tracked filenames. Since only the first hunk was staged, there are parts of `config.properties` that are staged and parts that are not staged.

Staged Timeout Change

1. Command: `git diff --staged`

2. Output:

```
diff --git a/config.properties b/config.properties
index f4c0461..34f139a 100644
--- a/config.properties
+++ b/config.properties
@@ -1,5 +1,5 @@
  # Core runtime settings
-timeout=30
+timeout=60
  retry.count=3
  retry.delay.ms=250
```

3. Explanation: Running `git diff --staged` compares the index against the commit at HEAD. The output confirms that only the timeout hunk was moved into the index to form the proposed snapshot for the next commit.

Unstaged Changes

1. Command: `git diff`

2. Output:

```
diff --git a/config.properties b/config.properties
index 34f139a..1cdalc5 100644
--- a/config.properties
+++ b/config.properties
@@ -10,7 +10,7 @@ connection.keepalive=true
  connection.pool.size=10

  # Logging settings
-log.level=INFO
+log.level=DEBUG
  log.console=true
  log.timestamps=true
  log.file=app.log
@@ -21,6 +21,6 @@ feature.metrics=false
```

```
feature.experimental=false

# Limits
-max.connections=100
+max.connections=250
max.payload.kb=512
max.queue.depth=25
```

3. Explanation: Executing `git diff` shows the remaining differences between the working tree and the index. This demonstrates that the logging configuration change remains intact in the working tree without being lost, discarded, or staged.

Part 3 — Explain Remote-Tracking State

Sequence: 1: In Alice's clone, create one local commit without pushing it.

1. Command: `echo "Alice change" >> README.md`
`git commit -am "Alice local change"`
`git log --graph --oneline --decorate --all`

2. Output:

- For `git commit....`:

```
[main 328d6d9] Alice local change
1 file changed, 1 insertion(+)
```

- For `git log...`

```
* 328d6d9 (HEAD -> main) Alice local change
* cc72c37 (origin/main) Update README
* bd77aab Add logging
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

3. Explanation: Creating a commit only updates objects and branch references inside the local `.git/` database. Alice's local `main` reference advances to point to the new commit, `328d6d9`, while her remote-tracking reference `origin/main` remains at `cc72c37`. This proves that committing locally modifies local state without updating the remote-tracking pointer. **Repository Changed: Alice's clone only**

Sequence: 2: Show that Alice's main moved while Alice's origin/main did not.

1. Command: `git rev-parse main origin/main`
2. Output:

```
328d6d95243274b5279e8065f5499c348d41b76e
cc72c37b04597b67198ca5c11085492da30f57f7
```

3. Explanation: Executing `git rev-parse` resolves the two reference names to their respective commit IDs. The output confirms that Alice's local `main` pointer has moved forward to her new commit object, whereas `origin/main` continues to point to the older commit. This demonstrates that `main` and `origin/main` are separate local references tracking different graph positions. **Repository Changed: None: Read Only**

Sequence: 3: Publish Alice's commit to the isolated remote.

1. Command: `git push origin main git log --graph --oneline --decorate --all`
2. Output:
 - For `git push...`:

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
```

```
Compressing objects: 100% (3/3), done.  
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s, do  
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0  
To /home/swearingen11/CPS-490-Capstone/ia-01-git-lab-stud  
cc72c37..328d6d9  main -> main
```

- For `git log...`:

```
* 328d6d9 (HEAD -> main, origin/main) Alice local change  
* cc72c37 Update README  
* bd77aab Add logging  
* 633624a Add configuration loader  
* 2b79f25 Initialize project
```

3. Explanation: Running `git push` transfers the new commit objects to `origin.git` and updates the remote repository's `main` reference. Upon receiving network confirmation, Git updates Alice's local `origin/main` reference to match her local `main` branch at `328d6d9`. **Repository Changed: The bare remote and Alice's clone**

Sequence: 4: Before Bob fetches, show that Bob's `origin/main` is still stale.

1. Command: `git log --graph --oneline --decorate --all`
2. Output:

```
* cc72c37 (HEAD -> main, origin/main) Update README  
* bd77aab Add logging  
* 633624a Add configuration loader  
* 2b79f25 Initialize project
```

3. Explanation: Even though Alice successfully pushed her commit to the bare remote, Bob's repository state remains completely unaffected. Bob's local `origin/main` still points to `cc72c37`, demonstrating that `origin/main` is a stale local cache rather

than an active network link into `origin.git`. **Repository Changed: None**

Sequence: 5: Fetch from origin in Bob's clone.

1. Command: `git fetch origin`
2. Output:

```
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reuse
Unpacking objects: 100% (3/3), 278 bytes | 278.00 KiB/s,
From /home/swearingenll/CPS-490-Capstone/ia-01-git-lab-st
    cc72c37..328d6d9  main        -> origin/main
```

3. Explanation: Executing `git fetch` initiates network communication with `origin.git` to download missing commit objects into Bob's repository. This operation updates Bob's local knowledge of the remote state without attempting to modify Bob's local working directory or branch references. **Repository Changed: Bob's clone only**

Sequence: 6: Show that Bob's origin/main moved while Bob's local main did not.

1. Command: `git log --graph --oneline --decorate --all`
2. Output:

```
* 328d6d9 (origin/main) Alice local change
* cc72c37 (HEAD -> main) Update README
* bd77aab Add logging
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

3. Explanation: The output shows that Bob's remote-tracking reference `origin/main` moved forward to commit `328d6d9`, while his local branch `main` remained at commit `cc72c37`. This confirms that `git fetch` updates local tracking references to reflect remote movement while leaving local branch pointers untouched. **Repository Changed: None**

Part 4 - Compare Merge and Rebase

Merge

Graph Before Merging

1. Command: `git log --graph --oneline --decorate --all`
2. Output:

```
* 0f9a8b4 (tag: D, feature/validation) Add validation tes
* 04ec968 (tag: C) Add timeout validation
| * cc72c37 (HEAD -> main, tag: F) Update README
| * bd77aab (tag: E) Add logging
|/
* 633624a (tag: B) Add configuration loader
* 2b79f25 (tag: A) Initialize project
```

3. Prediction: Merging `feature/validation` into `main` will join the two diverging histories without altering existing commits. Git will create a new merge commit `M` on `main` that contains two parent pointers—one to `cc72c37` (`main`) and one to `0f9a8b4` (`feature/validation`).

Merge Command

1. Command: `git merge --no-edit feature/validation`
2. Output:

```
Merge made by the 'ort' strategy.
src/config.sh | 10 ++++++++
tests/test-timeout.sh | 10 ++++++++
2 files changed, 20 insertions(+)
create mode 100755 tests/test-timeout.sh
```

Graph After Merging

1. Command: `git log --graph --oneline --decorate --all`

2. Output:

- For `git log...`:

```
* 52dd9b9 (HEAD -> main) Merge branch 'feature/validation'
|\
| * 0f9a8b4 (tag: D, feature/validation) Add validation t
| * 04ec968 (tag: C) Add timeout validation
* | cc72c37 (tag: F) Update README
* | bd77aab (tag: E) Add logging
|/
* 633624a (tag: B) Add configuration loader
* 2b79f25 (tag: A) Initialize project
```

- For `git cat-file...`:

```
tree 29071292c2ff4a64c117e0ae89a6c3376c4ebea5
parent cc72c37b04597b67198ca5c11085492da30f57f7
parent 0f9a8b45a43588f7ebbb7a6ca82786fc350ab813
author Git Lab <git-lab@example.com> 1789762752 -0400
committer Git Lab <git-lab@example.com> 1789762752 -0400
```

Merge branch 'feature/validation'

3. Inspecting the newly created merge commit with `git cat-file -p HEAD` displays two distinct parent header lines. The first parent line points to the pre-merge SHA of `main`

(cc72c37b04597b67198ca5c11085492da30f57f7), and the second parent line points to the SHA of feature/validation (0f9a8b45a43588f7ebbb7a6ca82786fc350ab813). This demonstrates that merging preserves existing branch histories and records their convergence by creating a single commit object with two parents.

Rebase

Graph Before Rebase

1. Command: `git log --graph --oneline --decorate --all`

2. Output:

```
* 0f9a8b4 (HEAD -> feature/validation, tag: D) Add valida
* 04ec968 (tag: C) Add timeout validation
| * cc72c37 (tag: F, main) Update README
| * bd77aab (tag: E) Add logging
|/
* 633624a (tag: B) Add configuration loader
* 2b79f25 (tag: A) Initialize project
```

3. Commit IDs:

- 0f9a8b4 -> Add validation tests
- 04ec968 -> Add timeout validation

Rebase Command

1. Command: `git rebase main`

2. Output: Successfully rebased and updated refs/heads/feature/validation.

Graph After Rebase

1. Command: `git log --graph --oneline --decorate --`

all

2. Output:

```
* 3584fbe (HEAD -> feature/validation) Add validation tes
* 46815ad Add timeout validation
* cc72c37 (tag: F, main) Update README
* bd77aab (tag: E) Add logging
| * 0f9a8b4 (tag: D) Add validation tests
| * 04ec968 (tag: C) Add timeout validation
| /
* 633624a (tag: B) Add configuration loader
* 2b79f25 (tag: A) Initialize project
```

3. Commit IDs:

- 3584fbe -> Add validation tests
- 46815ad -> Add timeout validation

4. Explanation: When executing a rebase, Git replays the changes onto a new base commit, preserving the human-readable commit message, author metadata, and file modifications. However, because the rebased commit is placed on top of a different base commit, its parent pointer changes. Because the parent pointer is one of the required header fields hashed to generate the commit ID, modifying the parent alters the calculated hash, producing a brand-new commit object with a distinct commit ID.

Summary: Merge vs. Rebase Comparison

- **Merge** preserves existing branch topology and historical chronology by joining two diverging lineages with a single, dual-parent commit.
- **Rebase** recreates feature work as a linear sequence descending from a new base commit, replacing the original feature commits with newly generated commit objects.

Part 5 — Rewrite a Development History

The four-commit history before the rewrite:

1. Command: `git log --oneline main..HEAD`
2. Output:

```
a8b2232 (HEAD -> feature/parser) remove debug output  
ecd2731 fix test  
4bf2afe oops add debug helper  
649da0a Implement parser
```

3. Explanation: Inspecting the branch history relative to `main` using `git log --oneline main..HEAD` displays four distinct commits. Rather than representing clean milestones, these commits record the temporary mechanics of trial-and-error development.

The transformation plan used:

1. Command: `git rebase -i HEAD~4`
2. Transformation Plan (in git editor):

```
pick 649da0a Implement parser  
fixup 4bf2afe oops add debug helper  
fixup ecd2731 fix test  
fixup a8b2232 remove debug output
```

3. Explanation: Executing `git rebase -i HEAD~4` launches Git's interactive rebase editor to specify a sequence transformation plan. Retaining `pick` on the initial commit (G) keeps it as the starting baseline, while marking the subsequent three commits (H, I, J) as `fixup` instructs Git to merge their diffs directly into G while discarding their separate commit messages.

The one-commit history after the rewrite:

1. Command: `git log --oneline main..HEAD`

2. Output: 1033a07 (HEAD -> feature/parser) Implement parser
3. Explanation: Running `git log --oneline main..HEAD` following the rebase confirms that the four developmental commits were successfully squashed into a single, clean commit object. This presents a coherent, single-purpose change suitable for peer review without noise from temporary fixes.

The result of running `bash ./tests/test-parser.sh` after the rewrite:

1. Command: `bash ./tests/test-parser.sh`
2. Output: `parser tests passed`
3. Explanation: * Executing the automated test script verifies that the consolidated commit snapshot retains the exact required functionality.

Why Behavioral Checks Are Required After History Rewrites

- An interactive rebase generates brand-new commit objects. Even when a rewritten commit graph looks visually clean and well-organized, squashing, reordering, or editing commits can accidentally introduce drop critical code adjustments or misapply hunks. Running an automated test script provides objective evidence that structural history cleanups have preserved intended software behavior.

Part 6 - Investigate and Resolve a Conflict

Produce evidence for the BASE, OURS, and THEIRS values of the timeout setting before merging:

BASE

1. Command: `git show $(git merge-base main feature/conflict):config.properties | grep timeout`
2. Output: `timeout=30`

OURS

1. Command: `git show main:config.properties | grep timeout`
2. Output: `timeout=60`

THEIRS

1. Command: `git show feature/conflict:config.properties | grep timeout`
2. Output: `timeout=DEFAULT_TIMEOUT`

Explanation:

3. Inspecting the content of `config.properties` across all three commit pointers reveals that both branches modified the exact same base value. OURS changed the setting to an explicit integer value (60), whereas THEIRS changed it to a symbolic constant (DEFAULT_TIMEOUT).

Attempt to merge feature/conflict into main and capture the resulting conflict state:

1. Command: `git merge feature/conflict`
2. Output:

```
Auto-merging config.properties
CONFLICT (content): Merge conflict in config.properties
Automatic merge failed; fix conflicts and then commit the
```

3. Conflict State Evidence:

- Using `git status`:

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
```

```
Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   config.properties
```

```
no changes added to commit (use "git add" and/or "git commit")
```

- Using `cat config.properties`:

```
# Core runtime settings
<<<<<< HEAD
timeout=60
=====
timeout=DEFAULT_TIMEOUT
>>>>>> feature/conflict
retry.count=3
retry.delay.ms=250
```

Explain what Git knows and what intent Git cannot infer:

A merge conflict is not a Git failure or a system error. Git successfully identified the common ancestor (`BASE`) and recognized that both `HEAD (OURS)` and `feature/conflict (THEIRS)` modified the exact same lines of code relative to that ancestor. Git halts the merge process because it cannot infer human intent to decide whether `60` or `DEFAULT_TIMEOUT` is the correct functional behavior for the application.

Completed Resolution

Edit `config.properties`

1. Command: `nano config.properties` then `cat`

```
config.properties
```

2. Output:

```
# Core runtime settings
HEAD
timeout=60
retry.count=3
retry.delay.ms=250
```

3. Explanation: Running `nano config.properties` opened the file, then I manually removed the conflict markers and the incorrect timeout, then I kept the line `timeout=60`.

Finalize the Resolution

Command: `git add config.properties` and `git merge --continue` Output: [main 0c61033] Merge branch 'feature/conflict'

Clean Repository and Intended Content

1. Command: `git status` and `git show`
HEAD:config.properties | grep timeout
2. Output:
 - For `git status`:

```
On branch main
nothing to commit, working tree clean
```

- For `git show...:timeout=60`
3. Explanation: Executing `git status` verifies that the repository has returned to a clean working state with no unmerged paths remaining. Inspecting the completed merge commit object confirms that the resolved file snapshot contains `timeout=60` as intended.

Part 7 - Recover Apparently Lost History

Record the initial tip of main and the commits that follow tag F.

1. Command: `git log --graph --oneline --decorate --all`
2. Output:

```
* 3ba526f (HEAD -> main) Add recovery marker
* 64e9b95 Enable demo mode
* f842947 Add recovery demo note
* cc72c37 (tag: F) Update README
* bd77aab Add logging
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

3. Explanation: Inspecting the commit graph prior to resetting shows that the `main` branch tip currently points to commit `3ba526f`. The commits `f842947`, `64e9b95`, and `3ba526f` represent the work added above tag `F`.

Move main back to F with a hard reset.

1. Command: `git reset --hard F`
2. Output: `HEAD is now at cc72c37 Update README`
3. Explanation: Executing `git reset --hard F` moves the `main` branch pointer back to the commit marked by tag `F`, while simultaneously synchronizing both the index and working tree to match that earlier snapshot.

Show that the later commits are no longer reachable from main and that `RECOVERY.txt` is no longer in the working tree.

1. Command: `git log --graph --oneline --decorate --all` and `ls RECOVERY.txt`
2. Output:

- for `git log...`:

```
* cc72c37 (HEAD -> main, tag: F) Update README
* bd77aab Add logging
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

- for `ls...:ls:` cannot access 'RECOVERY.txt': No such file or directory

3. Explanation: * Because the `main` branch reference was moved back to `cc72c37` and no other branch points to `3ba526f`, the later commits no longer appear in the reachable history graph. Additionally, the working directory was reset to tag `F`, causing `RECOVERY.txt` to disappear from the local filesystem.

Use the reflog to locate the old branch tip.

1. Command: `git reflog -5 --oneline`
2. Output:

```
cc72c37 (HEAD -> main, tag: F) HEAD@{0}: reset: moving to
3ba526f HEAD@{1}: commit: Add recovery marker
64e9b95 HEAD@{2}: commit: Enable demo mode
f842947 HEAD@{3}: commit: Add recovery demo note
cc72c37 (HEAD -> main, tag: F) HEAD@{4}: checkout: moving
```

3. Explanation: The `git reflog` command displays the local append-only log tracking every position `HEAD` has occupied in this repository. Entry `HEAD@{0}` records the hard reset operation that moved `main` to `cc72c37`, while entry `HEAD@{1}` captures the exact state immediately before the reset, revealing that the branch tip was commit `3ba526f`.

Create a branch named `rescue` that makes the old history reachable again.

1. Command: `git branch rescue HEAD@{1}` and `git log --graph --oneline --decorate --all`
2. Output: (`git log...`)

```
* 3ba526f (rescue) Add recovery marker
* 64e9b95 Enable demo mode
* f842947 Add recovery demo note
* cc72c37 (HEAD -> main, tag: F) Update README
* bd77aab Add logging
* 633624a Add configuration loader
* 2b79f25 Initialize project
```

3. Explanation: Executing `git branch rescue HEAD@{1}` creates a new named reference pointing directly to commit `3ba526f`. By pointing a new reference to `3ba526f`, the entire un-reset lineage becomes reachable again in the repository graph.

Prove that `RECOVERY.txt` can be read from the recovered history.

1. Command: `git show rescue:RECOVERY.txt`
2. Output: This file exists so we can apparently lose it with `reset --hard` and recover it using the `reflog`.
3. Explanation: Executing `git show rescue:RECOVERY.txt` extracts and displays the file blob directly from the snapshot referenced by `rescue`. This proves that the file content and historical objects remained fully intact in Git's object database throughout the reset.

Moving a Reference vs. Destroying Objects

Executing `git reset --hard` moves a branch reference to a different commit ID and updates the working tree, but it does not delete or

destroy commit objects in Git's database. Git commit objects remain stored on disk even when they are no longer pointed to by an active branch. Because Git records reference movement in the `reflog`, "lost" history can be recovered by locating the old commit's SHA hash and creating a new branch reference (such as `rescue`) to make those commits reachable again.

Part 8 — Search History with Bisect

The command(s) used to start the search

1. Command: `git bisect start bisect-bad bisect-good`
2. Output:

```
Bisecting: 4 revisions left to test after this (roughly 2
[72fa345e97e58040afc0652186f921717b015fdb] Refactor timeo
```

3. Explanation: Executing `git bisect start` initiates binary search mode by providing Git with two historical endpoints: `bisect-good` where the behavior worked, and `bisect-bad` where the behavior is broken. Git checks out a candidate commit near the midpoint of this range to test whether the bug was introduced before or after that commit.

Evidence from the automated search identifying the first bad commit

1. Command: `git bisect run bash ./tests/test-compute.sh`
2. Output:

```
running 'bash' './tests/test-compute.sh'
Bisecting: 2 revisions left to test after this (roughly 1
[0e5e6f5358c1f9ee6ed576f3fb0103db4e004c64] Document pre-b
```

```
running 'bash' './tests/test-compute.sh'
Bisecting: 0 revisions left to test after this (roughly 1
[bf17fa3b3d9f21bbf9061415287fd400af59ba25] Document pre-b
running 'bash' './tests/test-compute.sh'
72fa345e97e58040afc0652186f921717b015fdb is the first bad
commit 72fa345e97e58040afc0652186f921717b015fdb
Author: Git Lab <git-lab@example.com>
Date:   Mon Sep 14 10:28:44 2026 -0400
```

```
Refactor timeout normalization
```

```
src/compute.sh | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first bad commit
```

3. Explanation: Executing `git bisect run` automates the search by repeatedly checking out candidate commits and evaluating the classifier script. The script's exit status classifies each commit: an exit code of 0 indicates `good`, while any non-zero failure exit status indicates `bad`. Git uses each result to narrow the search range until it isolates the exact commit where the failure first appeared.

First Bad Commit ID and Subject

- First Bad Commit ID:
72fa345e97e58040afc0652186f921717b015fdb
- Subject Line: Refactor timeout normalization
- Explanation: The automated bisect session locates the precise commit where the test classifier transitioned from passing to failing. This identifies the single commit snapshot responsible for introducing the regression.

End the Bisect Session Cleanly

1. Command: `git bisect reset`
2. Output:

```
Previous HEAD position was bfl7fa3 Document pre-bug chang  
Switched to branch 'demo/bisect'
```

3. Explanation: Executing `git bisect reset` terminates the bisect session and returns both `HEAD` and the working directory to the original branch state prior to starting the search.

Exit Status Classification and Binary Search Efficiency

- **Exit Status Classification:** When `git bisect run` executes an automated test script, it interprets the process exit code to classify the evaluated commit. An exit status of 0 signals success (good), prompting Git to move the known-good boundary forward. Any non-zero exit status signals failure (bad), prompting Git to move the known-bad boundary backward.
- **Binary Search vs. Sequential Search Efficiency:** A sequential search checks commits one by one, requiring N total test evaluations for N candidate commits. In contrast, binary search tests a commit at the midpoint of the search range, eliminating roughly half of the remaining candidates with every test. Because the search space shrinks exponentially ($O(\log_2 N)$), evaluating a history of 1,024 candidate commits requires only about 10 test evaluations in the ideal case.

Reflection

1. What is the difference between a commit and a branch?

A commit is an immutable object containing a directory tree snapshot, metadata, and backward parent pointer(s) that record historical state. A branch is simply a movable, named reference that identifies a specific commit position within that history graph.

2. What role does the index play in constructing the next commit?

The index holds the staged snapshot of proposed content between the working tree and `HEAD`. It allows developers to deliberately construct the next commit snapshot ensuring each commit records one coherent change rather than an indiscriminate dump of working tree edits.

3. Why is origin/main local state, and what exactly does git fetch change?

`origin/main` is a local reference stored inside `.git/refs/remotes/` that records the remote branch state last observed, rather than an active network connection to a remote server. Running `git fetch` downloads missing commit objects and updates `origin/main` to reflect remote movement, without altering local branch references or modifying working tree files.

4. How does a merge differ structurally from a rebase?

A merge joins diverging histories by creating a brand-new integration commit with two parent pointers, preserving existing commit identities and branch topology. A rebase replays patch changes sequentially onto a new base, generating brand-new commit objects with distinct hashes and new parent pointers to form a linear history.

5. What information is Git missing when it reports a merge conflict?

Git detects that both branches modified the exact same base content relative to a common ancestor. Git halts because it lacks final human and product intent. It cannot infer application logic to determine which resulting snapshot or behavior is desired.

6. Why can the reflog sometimes recover history after a destructive-looking reset?

Executing `git reset --hard` moves a branch reference and updates the working tree, but it does not delete or destroy the underlying commit objects in the repository database. Because the `reflog` maintains a local append-only record of every position `HEAD` has occupied, locating the pre-reset commit ID and giving it a new reference name restores

graph reachability to the "lost" history.

7. What makes an automated test useful to git bisect?

An automated test script acts as a classifier using its exit status to evaluate candidate commits. This allows `git bisect run` to automate a binary search that eliminates roughly half of the remaining candidate commits with each test evaluation.

8. What common mental model connects merge, rebase, reset, reflog, and bisect?

The unifying mental model is a Directed Acyclic Graph (DAG) of commit objects, navigated by movable references, and managed across three distinct state boundaries (working tree, index, and commit at HEAD)